

Università degli Studi di Cagliari

FACOLTÀ DI INGEGNERIA E ARCHITETTURA

CORSO DI LAUREA TRIENNALE IN INGEGNERIA ELETTRICA, ELETTRONICA E INFORMATICA

Agenti AI per la Generazione di Codice

RELATORE
Dott.ssa Ambra Demontis

CANDIDATO
Marco Cogoni

A.A 2025/2026

Abstract

L'avvento dei Large Language Models (LLM) ha rivoluzionato l'Ingegneria del Software, cambiando la generazione di codice da un problema di sintesi formale a un'operazione dinamica e assistita. Tuttavia, l'uso di modelli tradizionali — che tentano di generare la soluzione in un unico tentativo — su progetti reali evidenzia limiti strutturali critici, ad esempio: l'incapacità di gestire interi repository, le allucinazioni logiche e l'assenza di meccanismi di autocorrezione in autonomia.

Per superare queste criticità, nella tesi analizziamo l'evoluzione verso gli ecosistemi di Agenti di Intelligenza Artificiale, che utilizzano una revisione sistematica della letteratura. L'elaborato confronta architetture ad agente singolo e framework cooperativi multi-agent. Dimostreremo come la suddivisione dei ruoli, l'integrazione di interfacce su misura (Agent-Computer Interfaces) e il passaggio al debugging dinamico a runtime incrementino radicalmente l'affidabilità del software generato.

Infine, lo studio valuta criticamente: il paradosso della produttività indotto da queste tecnologie, l'accumulo silente di debito tecnico, le persistenti vulnerabilità di sicurezza introdotte e le discrepanze dei benchmark statici odierni, dimostrando oggettivamente che, allo stato dell'arte, l'intervento umano rimane una componente imprescindibile per il mantenimento architettonico e strutturale dei sistemi software.

Indice

1 Introduzione **1**

1.1 Introduzione ai Large Language Models 1

2 Large Language Models e Agenti AI **5**

2.1 L'evoluzione: dai Large Language Models agli Agenti 5

2.2 Tassonomia e Architettura Logica 8

2.3 Il Paradigma Multi-Agente nell'Ingegneria del Software 10

3 Strumenti per gli Agenti **14**

3.1 L'Interfaccia Agente-Computer (ACI) 14

3.2 Navigazione e Strumenti a Livello di Repository 16

3.3 Pianificazione a Grafo e Gestione Dinamica dei Dati 19

4 L'Evoluzione dell'Affidabilità: Testing e Debugging Dinamico **24**

4.1 Il Paradigma Self-Improving: L'Apprendimento senza Pesi 24

4.2 Separazione dei Ruoli e Isolamento dei Task 26

4.3 Debugging a Runtime: Dall'Analisi Statica all'Esecuzione Attiva . . 27

4.4 Automazione della Code Review 29

4.5 Sintesi Architetturale: Un Ecosistema di Apprendimento Integrato . 31

5 Il Paradosso della Produttività: Limiti e Sicurezza **34**

5.1 Il Paradosso dei File di Contesto 34
5.2 Agentic Refactoring: Pulizie di Superficie o Ristrutturazione? . . . 36
5.3 L'Illusione del Prompting per la Sicurezza 38
5.4 L'Eredità dell'IA: Debito Tecnico e Manutenzione 40

6 Il Problema della Valutazione (Benchmarking) 43
6.1 I Limiti dei Test Attuali: Contaminazione e Irrealtà 43
6.2 Generazione Automatica di Scenari di Valutazione 45
6.3 Verso Benchmark Dinamici: La Realtà delle Prestazioni 46

7 Conclusioni 49

Capitolo 1

Introduzione

1.1 Introduzione ai Large Language Models

L'avvento e la rapida affermazione dei *Large Language Models* (LLM) hanno segnato un punto di svolta decisivo, trasformando radicalmente il panorama dell'Intelligenza Artificiale e dell'Ingegneria del Software [1]. Da un punto di vista architetturale, gli LLM sono una classe di modelli linguistici pre-addestrati fondati su tecnologie di *deep learning*, che sfruttano prevalentemente l'architettura *Transformer* come nucleo strutturale [13]. Nonostante i risultati generati possano apparire dotati di raziocinio, il meccanismo fondamentale su cui si basano è strettamente probabilistico: il loro compito primario consiste nell'eseguire un calcolo matematico per determinare quale sia la parola (o *token*) con la maggiore probabilità di seguire quelle precedenti all'interno di una data sequenza. Questo processo computazionale si fonda sull'apprendimento di pattern statistici e relazioni contestuali del linguaggio attraverso una massiccia fase di pre-addestramento non supervisionato su corpus testuali di dimensioni colossali [8]. Nel contesto specifico dell'informatica, l'esposizione durante l'addestramento a immensi archivi di codice *open-source* permette a questi modelli di calcolare in modo accurato le probabilità

delle sequenze sintattiche proprie di molteplici linguaggi di programmazione, sostituendo di fatto la comprensione concettuale con la predizione statistica. Sebbene operino a un livello puramente predittivo, privo di un'autentica intelligenza umana, la combinazione di miliardi di parametri e set di dati onnicomprensivi ha posto le solide basi tecnologiche necessarie per il successivo salto evolutivo dell'Ingegneria del Software verso l'autonomia [1, 8].

Struttura della tesi. La presente tesi è organizzata come segue. In questo capitolo, abbiamo fornito un'introduzione ai *Large Language Models* (LLM), delineando la loro evoluzione dai modelli passivi ai moderni ecosistemi di Agenti AI ed esplorandone l'architettura logica e il paradigma multi-agente. Il Capitolo 2 analizza le modalità con cui questi agenti interagiscono con l'ambiente operativo, esaminando le Interfacce Agente-Computer (ACI), gli strumenti di navigazione a livello di repository e le tecniche di pianificazione dinamica a grafo. Il Capitolo 3 si concentra sull'evoluzione dell'affidabilità del codice generato, descrivendo i paradigmi di auto-miglioramento, la rigida separazione dei ruoli nel *testing* e le metodologie avanzate di *debugging* e revisione dinamica a *runtime*. Nel Capitolo 4 affrontiamo il "paradosso della produttività", mettendo in luce i limiti operativi e le criticità legate alla sicurezza. Tra questi, esaminiamo l'impatto controintuitivo dei file di contesto (documenti testuali inseriti all'interno dei repository per fornire all'agente linee guida e istruzioni specifiche sul progetto [2]), le carenze nel *refactoring* autonomo e l'accumulo silente di debito tecnico, definibile come il costo implicito delle future manutenzioni causato dall'adozione di soluzioni implementative rapide ma subottimali [7]. Il Capitolo 5 esamina le problematiche relative alla valutazione (*benchmarking*) degli agenti, evidenziando le limitazioni e la contaminazione dei test statici attuali e presentando nuovi framework per la generazione automatica di benchmark dinamici. Infine, nel Capitolo 6, discutiamo le conclusioni tratte da questo studio e delineiamo le prospettive future dell'intelligenza

artificiale applicata all'Ingegneria del Software.

Capitolo 2

Large Language Models e Agenti AI

2.1 L'evoluzione: dai Large Language Models agli Agenti

Fino a pochi anni fa, i Large Language Models (LLM) – modelli probabilistici basati su reti neurali profonde, addestrati su enormi corpus testuali –operavano come strumenti puramente reattivi e conversazionali: sistemi isolati, dotati di una vasta base di conoscenza, ma limitati all'interazione tramite interfaccia chat e capaci unicamente di fornire risposte testuali agli utenti. L'introduzione degli Agenti di Intelligenza Artificiale ha superato i limiti dell'assistenza passiva (ovvero la modalità in cui il modello si limita a ricevere un input e produrre un output in un singolo passaggio, senza pianificazione attiva o interazione continua con l'ambiente [1]), dotando questi sistemi di capacità di interazione diretta con gli ambienti di sviluppo e le risorse di sistema. Fornendo ai modelli linguistici l'accesso diretto a terminali, compilatori e file system, si è segnato il passaggio definitivo dalla semplice elaborazione teorica all'azione autonoma.

L'Ingegneria del Software tratta la generazione automatica di codice come una

sfida storica (poiché automatizzare questo processo rappresenta da sempre il traguardo fondamentale per incrementare la produttività dello sviluppo [1]). L'obiettivo pratico è tradurre le intenzioni umane, fornite tramite specifiche formali del software o linguaggio naturale, in programmi eseguibili. Inizialmente, la ricerca si concentrava sulla sintesi formale dei programmi (un approccio volto a derivare programmi verificabilmente corretti basandosi su specifiche logiche fornite a priori) al fine di creare un software formalmente corretto. Tuttavia, la stesura manuale delle specifiche formali si rivelava estremamente complessa; di conseguenza, l'approccio basato sulla deduzione formale è rimasto limitato a domini applicativi molto ristretti [1].

Per ottenere un maggiore livello di astrazione, il focus della ricerca scientifica nel settore si è spostato sui modelli guidati dai dati (data-driven) basati sull'apprendimento profondo (deep learning). Attraverso questa transizione, la produzione di codice è stata riformulata come un problema di comprensione e generazione di sequenze probabilistiche. Tuttavia, i primi approcci di deep learning applicati alla stesura di codice presentavano diverse restrizioni: generavano frequentemente errori sintattici o semantici, causando il blocco delle compilazioni o delle esecuzioni del programma [1].

L'introduzione dei Large Language Models ha permesso di superare le limitazioni dei modelli precedenti. Concepiti originariamente per l'Elaborazione del Linguaggio Naturale (NLP - Natural Language Processing), gli LLM trovano oggi un'applicazione sistematica nell'ingegneria del software. La loro efficacia dipende direttamente dai dataset utilizzati durante la fase di pre-addestramento: l'esposizione a repository su larga scala, in gran parte tratti da piattaforme open-source come GitHub, consente ai modelli di assimilare la sintassi, la semantica e i costrutti algoritmici dei vari linguaggi di programmazione [1].

Sebbene la generazione di codice in modalità "single-response" (una modalità

di interazione in cui il modello linguistico produce un singolo output statico a fronte di un prompt testuale, senza iterazioni) garantisca buone prestazioni, l'impiego degli LLM nativi nello sviluppo di software strutturato presenta evidenti limitazioni. Il modello linguistico standard – inteso come l'architettura di base del modello, priva di strumenti esterni o flussi decisionali ciclici – non è infatti in grado di scomporre autonomamente attività complesse (task), non interagisce con gli ambienti di sviluppo e non esegue la validazione del codice, essendo sprovvisto di procedure per l'auto-correzione. Risulta pertanto inefficace l'utilizzo di tali sistemi per operazioni che richiedono l'analisi simultanea di più file di progetto (cross-file), la risoluzione dei difetti a runtime (debugging dinamico) o l'ottimizzazione iterativa del codice [1].

Gli Agenti di Intelligenza Artificiale basati su LLM sono stati progettati per risolvere queste specifiche criticità, adottando un'infrastruttura in cui il modello linguistico assume il ruolo di motore decisionale centrale. Tale architettura permette di costruire entità in grado di pianificare, agire e osservare i risultati, iterando le operazioni in totale autonomia. Rispetto alle tecniche di generazione precedenti, gli agenti introducono tre innovazioni principali [1]:

- **Autonomia:** Mentre un modello conversazionale tradizionale fornisce un'assistenza passiva, limitandosi ad azioni come l'autocompletamento del codice, un agente AI amministra proattivamente il flusso di lavoro. La gestione si estende dall'analisi dei requisiti fino all'implementazione pratica, relegando lo sviluppatore umano a un ruolo di supervisione del processo [1].

- **Estensione dello scope operativo (Task scope):** Gli agenti AI superano la mera produzione del singolo frammento di codice per operare sull'intero ciclo di vita dello sviluppo software (SDLC - Software Development Life Cycle). Sono in grado di analizzare requisiti incerti, strutturare l'ar-

chitettura del progetto, implementare suite di test e applicare operazioni di riorganizzazione strutturale e pulizia del codice (refactoring) [1].

- **Focus ingegneristico:** L'attenzione progettuale si sposta dall'accuratezza sintattica del singolo algoritmo all'affidabilità dell'intero sistema software. In questo modo, gli agenti conducono processi complessi e comunicano in modo efficiente con strumenti operativi (tool) e ambienti di esecuzione esterni [1].

Ma come è possibile, dal punto di vista strettamente ingegneristico, che un modello concepito per la sola predizione statistica del testo si trasformi in un attore digitale dotato di tale autonomia operativa? La risposta non risiede in un semplice incremento parametrico del modello linguistico stesso, bensì in un drastico cambio di paradigma infrastrutturale: l'LLM smette di operare come un generatore isolato e diventa il motore decisionale centrale di un sistema più ampio, strutturato in moduli specializzati.

2.2 Tassonomia e Architettura Logica

L'infrastruttura di un agente AI basato su LLM va ben oltre la semplice invocazione statica di un'Interfaccia di Programmazione delle Applicazioni (API). Da una prospettiva architettonica, infatti, un agente incorpora moduli interconnessi progettati per emulare i processi cognitivi umani.

Il nucleo operativo di un agente si fonda sull'integrazione delle capacità generative e di ragionamento (reasoning) del Large Language Model con moduli specializzati di percezione, memoria, processo decisionale e azione. I componenti fondamentali di tale infrastruttura includono [1]:

1. **Pianificazione (Planning):** La capacità di scomporre un problema computazionale complesso in sotto-obiettivi più piccoli e gestibili. Questo modulo permette all'agente di affrontare la risoluzione di problemi articolati in

sotto-attività sequenziali (multistep), caratteristica intrinseca della scrittura del software.

2. **Memoria (Memory):** Solitamente architettata in due livelli. La memoria a breve termine mantiene lo stato corrente dell'esecuzione e le istruzioni temporanee entro i limiti della finestra di contesto del LLM (ovvero la quantità massima di token testuali che il modello può elaborare simultaneamente in un dato istante). La memoria a lungo termine supera le limitazioni di capacità del modello originario sfruttando framework come la *Retrieval-Augmented Generation* (RAG) – una tecnica che ottimizza le risposte del modello recuperando dinamicamente informazioni esterne – per l'archiviazione persistente e il recupero dell'esperienza pregressa, avvalendosi di database vettoriali (strutture dati ottimizzate per archiviare e ricercare informazioni sotto forma di rappresentazioni matematiche, o *embedding*).

3. **Riflessione (Reflection):** Un modulo critico che consente all'agente l'analisi critica, la valutazione e la correzione autonoma dei propri contenuti generati. Questo meccanismo migliora iterativamente le decisioni passate, attuando un perfezionamento continuo in assenza di segnali di supervisione espliciti da parte dell'utente.

4. **Uso degli Strumenti (Tool Usage):** L'interazione con ambienti digitali esterni. Poiché i modelli linguistici standard sono sistemi intrinsecamente isolati, il modulo di utilizzo degli strumenti espande enormemente il loro spazio operativo, garantendo all'agente i permessi per l'invocazione di API di terze parti, l'esecuzione di codice tramite compilatori, l'accesso a interfacce a riga di comando e, infine, l'interrogazione diretta di motori di ricerca web.

Dal punto di vista della complessità compositiva e dell'interazione, la letteratura scientifica classifica le architetture in sistemi a singola entità (*Single-Agent*) e

sistemi cooperativi (*Multi-Agent*) [1]. Nello specifico, un'architettura *Single-Agent* opera come un sistema centralizzato in cui un unico modello linguistico porta a termine autonomamente i task assegnati. Tuttavia, al crescere della complessità del software da sviluppare, le limitazioni cognitive e computazionali di un singolo nodo decisionale portano spesso all'accumulo di difetti logici e vulnerabilità critiche. Per risolvere problemi ingegneristici su vasta scala, la ricerca si è orientata verso architetture *Multi-Agent*, in cui team composti da molteplici LLM (ai quali vengono assegnati ruoli specifici, ad esempio: analista dei requisiti, programmatore, tester di qualità) collaborano e negoziano interattivamente per raggiungere un obiettivo software comune [1].

2.3 Il Paradigma Multi-Agente nell'Ingegneria del Software

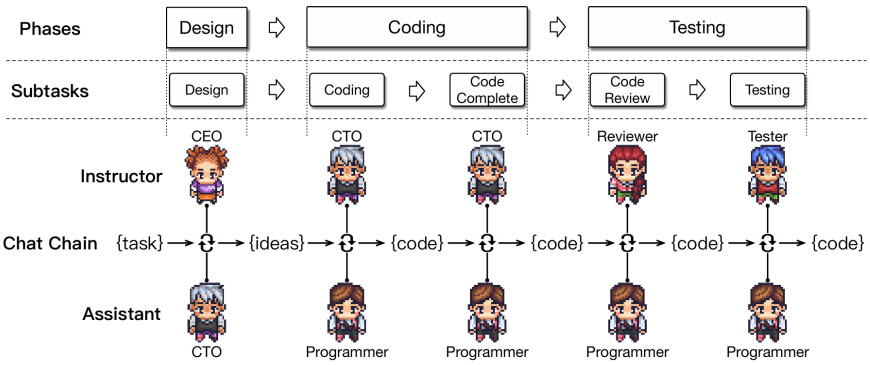


Figura 2.1: Workflow a catena (Chat Chain) del framework ChatDev (Fonte: [8])

Il passaggio verso sistemi multi-agente (*Multi-Agent*) è strettamente allineato con l'evoluzione storica dell'Ingegneria del Software, la quale ha progressivamente abbandonato la programmazione individuale in favore dello sviluppo collaborativo in team [1]. Lo sviluppo del software contemporaneo è, per sua natura, un processo

altamente complesso che necessita della cooperazione di molteplici figure professionali dotate di competenze eterogenee, quali architetti del software, programmatori e ingegneri addetti al collaudo (*testing*) [1].

Storicamente, l'applicazione dell'apprendimento profondo (*deep learning*) nell'Ingegneria del Software si è circoscritta al miglioramento di fasi specifiche e isolate del modello a cascata (*waterfall* – un approccio metodologico di tipo lineare e sequenziale allo sviluppo software). Tale frammentazione tecnologica, in cui ogni singola tappa, dalla raccolta dei dati all'inferenza, richiedeva architetture uniche, ha generato incongruenze tecniche tra i vari stadi, traducendosi in un processo di sviluppo disarticolato e inefficace [8].

Per porre rimedio alle incongruenze del modello a cascata, la ricerca scientifica ha introdotto approcci pionieristici basati sul linguaggio naturale come ponte unificatore per la risoluzione autonoma dei compiti. Un esempio prominente è *ChatDev*, un *framework* di sviluppo software basato su chat in cui molteplici agenti virtuali, guidati da LLM, assumono ruoli sociali e decisionali specializzati (ad esempio *Chief Executive Officer*, *Chief Technology Officer*, revisore e collaudatore). Questi agenti collaborano attivamente lungo le fasi fondamentali del ciclo di vita del software: progettazione, stesura del codice e verifica [8].

La principale innovazione architettonica del *framework ChatDev*, illustrata nella Figura 2.1, risiede nella cosiddetta *Chat Chain* (catena di conversazione). Tale paradigma suddivide ulteriormente le macro-fasi dello sviluppo software in sotto-attività (*sub-task*) granulari e gestibili. All'interno di ogni singola sotto-attività, due agenti interloquiscono assumendo le dinamiche di istruttore (*Instructor*) e assistente (*Assistant*). Questa struttura a catena fornisce agli agenti direttive esplicite sulle modalità di comunicazione e promuove la collaborazione, collegando in modo fluido sotto-attività definite in linguaggio naturale con sotto-attività tradotte in linguaggio di programmazione [8].

Una delle criticità primarie nelle architetture basate su LLM è costituita dalle cosiddette "allucinazioni da codice" (la generazione di codice sorgente incompleto, non eseguibile o sintatticamente impreciso, che fallisce nell'esaudire i requisiti originari e richiede onerosi interventi manuali). Per minimizzare tale fenomeno, il paradigma *ChatDev* implementa un meccanismo di *de-allucinazione comunicativa*. All'interno di questo modello di comunicazione si verifica una deliberata inversione dei ruoli: l'agente assistente richiede proattivamente all'agente istruttore dettagli più specifici e mirati prima di fornire una risposta definitiva (ad esempio, interrogandolo sul nome esatto di una dipendenza esterna). Questo scambio autorizza un trasferimento di informazioni ad altissima granularità (*fine-grained*) che ottimizza la soluzione e riduce drasticamente le allucinazioni da codice [8].

Gli studi condotti sulle architetture multi-agente hanno dimostrato che l'utilizzo del linguaggio naturale risulta estremamente vantaggioso per la progettazione del sistema a livello di astrazione logica, mentre la comunicazione diretta tramite linguaggio di programmazione si rivela essenziale ed efficace nelle fasi di *debugging* [8]. In conclusione, l'interazione *Multi-Agent* si sta affermando come il vettore principale per attenuare le vulnerabilità del software generato sinteticamente, garantendo una maggiore completezza, eseguibilità e aderenza ai requisiti aziendali originari [8].

Capitolo 3

Strumenti per gli Agenti

3.1 L'Interfaccia Agente-Computer (ACI)

Si immagini di dover interagire con un sistema operativo percependo esclusivamente flussi di testo grezzo, privi di astrazioni visive: questa è l'esperienza operativa standard di un modello linguistico. Per ottimizzare le prestazioni di tali modelli, non è sufficiente incrementarne unicamente le capacità cognitive, ma risulta fondamentale progettare un'interfaccia su misura, in modo analogo a come gli operatori umani si avvalgono di interfacce grafiche utente (GUI) per ovviare alla scrittura manuale di istruzioni a basso livello.

Storicamente, per la risoluzione di compiti di programmazione, gli agenti basati su *Large Language Models* (LLM) sono stati progettati per interfacciarsi direttamente con gli applicativi preesistenti, quali la shell di Linux o l'interprete Python. Tuttavia, nell'affrontare attività complesse come l'ingegneria del software, i programmatori umani traggono un enorme vantaggio dall'utilizzo di Ambienti di Sviluppo Integrati (IDE, *Integrated Development Environment*), piattaforme che forniscono astrazioni visive e strumenti avanzati per facilitare il *debugging* e la navigazione del codice. I modelli linguistici rappresentano a tutti gli effetti una

nuova categoria di utenti digitali, dotati di notevoli capacità generative ma soggetti a rigidi limiti cognitivi; tra questi spicca la finestra di contesto (*Context Window*), ovvero la quantità massima di informazioni elaborabile in una singola operazione. Per tale ragione, questi modelli faticano a operare in modo affidabile all'interno di ambienti originariamente concepiti per l'essere umano. A titolo esemplificativo, una tipica shell Linux offre comandi estremamente granulari e configurabili (come `grep` o `sed`), ma spesso opera in modo silente, non fornendo un *feedback* esplicito sugli effetti di una modifica o restituendo messaggi di errore poco diagnostici in caso di parametri complessi mal formulati. Tale mancanza innesca frequentemente *loop* di comandi inefficaci, portando l'agente a saturare la propria finestra di contesto con output testuali prolissi, irrilevanti e non corretti.

Per superare questo disallineamento tra le interfacce tradizionali e i limiti cognitivi degli agenti, è stato introdotto il concetto di *Agent-Computer Interface* (ACI), un livello di astrazione interposto tra il modello linguistico e il sistema operativo. L'obiettivo dell'ACI è fornire all'agente un set ridotto di comandi semplici, compatti ed efficienti, progettati specificamente per mitigarne i punti deboli. Un'implementazione allo stato dell'arte di questo paradigma è rappresentata dal framework *SWE-agent*[14], il quale sostituisce la tradizionale riga di comando con un'interfaccia personalizzata per la navigazione, la ricerca e la modifica dei file. Invece di vincolare l'agente all'utilizzo di complessi comandi Bash per la manipolazione testuale, l'ACI di *SWE-agent* espone un dispositivo interattivo di visualizzazione dei file (*File Viewer*), come visibile in Figura 3.1, che mostra blocchi limitati a 100 righe di codice per volta, corredate dai rispettivi numeri di riga. In questo modo, l'agente può analizzare il sorgente e invocare un comando di `edit` mirato per sostituire sezioni specifiche, eliminando la necessità di riscrivere interi file o calcolare complessi *offset* spaziali [14].

Inoltre, un'interfaccia rigorosa deve integrare meccanismi di sicurezza (*guar-*

drails) per prevenire la propagazione degli errori. Poiché gli LLM sono inclini a commettere inesattezze nell'indentazione o a introdurre difetti sintattici durante la modifica del codice, l'ACI di *SWE-agent* incorpora un linter automatico, ovvero uno strumento di analisi statica progettato per rilevare errori di formattazione o sintassi prima dell'esecuzione vera e propria sul *repository*. Qualora l'agente generi un comando errato, l'interfaccia blocca l'operazione e restituisce un *feedback* esplicito, mostrando sia il codice originale sia l'anomalia rilevata, forzando così il modello ad auto-correggersi. L'integrazione di tale architettura ACI, unita a un sistema di gestione dinamica che comprime gli output dei comandi pregressi per prevenire il sovraccarico informativo, trasforma quindi il modello linguistico in uno sviluppatore autonomo altamente performante, abbattendo drasticamente i cicli di fallimento iterativo [14].

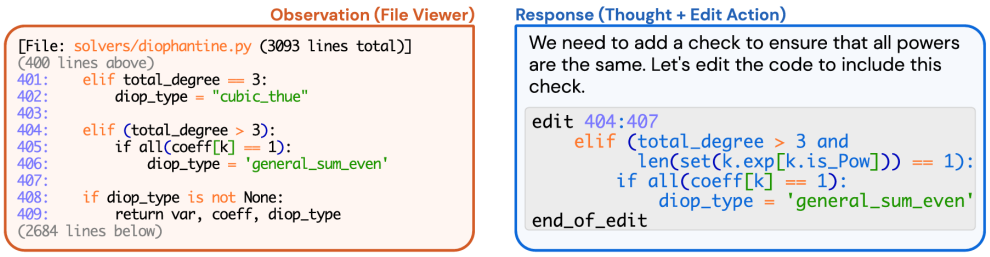


Figura 3.1: Integrazione tra File Viewer e interfaccia di modifica in SWE-agent (Fonte: [14])

3.2 Navigazione e Strumenti a Livello di Repository

Se a un idraulico viene richiesto di riparare una singola tubatura, l'ispezione locale del guasto risulta sufficiente. Tuttavia, qualora fosse necessario riprogettare la rete idrica di un intero grattacielo, diverrebbe indispensabile l'accesso alla planimetria

completa e a strumenti avanzati per comprendere le interconnessioni tra i vari condotti. Nell'Ingegneria del Software si presenta una dicotomia architetturale analoga: la scrittura di una singola funzione isolata è un'operazione ormai alla portata dei modelli intelligenti, ma la modifica di un intero progetto software richiede l'adozione di strumenti che permettano all'agente di esplorare l'ambiente di lavoro nella sua totalità, evitando il sovraccarico informativo.

Sebbene i *Large Language Models* (LLM) abbiano dimostrato notevoli capacità nella generazione automatica di codice, solitamente eccellono solo in scenari a bassa complessità. Tali scenari includono la scrittura di frammenti di codice indipendenti (*statement-level*, ovvero a livello di singola istruzione) o funzioni isolate (*function-level*, a livello di blocco logico indipendente). Lo sviluppo di sistemi software reali, tuttavia, implica l'interazione con complessi *repository* di codice, all'interno dei quali le funzioni presentano intricate dipendenze contestuali distribuite su molteplici file. Di conseguenza, per consentire agli LLM di affrontare sfide autentiche a livello di progetto (*repo-level*, ovvero agendo sull'intero archivio di codice condiviso), risulta necessario integrare nel loro processo decisionale l'utilizzo di strumenti di programmazione esterni, simulando fedelmente le pratiche analitiche adottate dagli sviluppatori umani [15].

Per superare questa limitazione legata all'assenza di contesto globale, è stato sviluppato *CodeAgent* (un framework da non confondere con omonime architetture orientate esclusivamente alla *code review*). Questa soluzione dota l'LLM di cinque strumenti specifici, suddivisi in tre categorie operative: il recupero delle informazioni (*Information Retrieval*, ovvero l'estrazione di dati pertinenti da fonti non strutturate), l'implementazione del codice e il *testing*. Durante la fase di recupero delle informazioni, l'agente può avvalersi di strumenti di *Website Search* per estrapolare soluzioni tramite motori di ricerca esterni e di *Documentation Reading* per interrogare la documentazione interna al progetto [15]. Il nucleo operativo del

sistema, indispensabile per agire a livello di *repository*, è tuttavia rappresentato dallo strumento di *Code Symbol Navigation*. Quest'ultimo è basato su analizzatori statici come *tree-sitter* (un generatore di parser incrementale utilizzato per costruire alberi sintattici volti all'analisi statica). Tale strumento permette all'agente di ispezionare il codice sorgente estraendo dinamicamente nomi di classi, variabili globali e firme di funzioni, navigando così l'albero delle dipendenze per comprendere le interazioni tra i moduli software prima ancora di generare una singola riga di codice [15].

Affinché questa architettura di strumenti esterni operi con efficacia, nello studio su CodeAgent gli autori hanno implementato e valutato specifiche strategie di interazione per guidare l'agente [15].Tra queste figurano il paradigma ReAct (Reasoning and Acting, un approccio in cui il modello alterna iterativamente deduzioni logiche all'esecuzione di azioni nell'ambiente) e l'utilizzo basato su regole (*Rule-based*, ovvero una strategia che definisce a priori una sequenza ordinata e specifica per l'invocazione degli strumenti, imitando il flusso di lavoro logico di un programmatore umano). Applicando tali strategie, l'agente apprende primariamente il contesto attraverso la navigazione tra i simboli, procede alla sintesi del codice e, infine, invoca in autonomia un *Format Checker* e un *Code Interpreter* per testarne l'esecuzione all'interno del *repository*.L'analisi empirica sull'impiego di questi strumenti, svolta nello studio di valutazione del framework CodeAgent [15], dimostra che il modulo di navigazione dei simboli è quello richiamato con la maggiore frequenza. Tale evidenza certifica che la profonda comprensione dell'architettura di un *repository* costituisce il prerequisito fondamentale per abilitare l'intelligenza artificiale alla generazione di codice coerente e affidabile all'interno di progetti software su larga scala.

3.3 Pianificazione a Grafo e Gestione Dinamica dei Dati

Nella gestione di progetti complessi, la capacità di riadattamento dinamico di fronte a imprevisti logici o sui dati è cruciale. Proprio per rispondere a questa necessità, è stato proposto un innovativo tipo di pianificazione a grafo concepito specificamente per la creazione di programmi che si occupano di Data Science. Mentre i sistemi tradizionali di Intelligenza Artificiale eseguivano sequenze di istruzioni rigide, arrestandosi al primo errore, i framework contemporanei che adottano questo approccio emulano le capacità di pianificazione di un project manager: organizzano i compiti in strutture topologiche e, in caso di anomalie, riprogrammano dinamicamente i passaggi successivi in tempo reale.

Oltre all'Ingegneria del Software tradizionale, l'applicazione degli agenti autonomi si espande rapidamente verso scenari di *Data Science*, i quali presentano sfide architetturali uniche. A differenza della pura stesura di codice, il flusso di lavoro dell'analisi dei dati richiede la gestione di *task* interconnessi a lungo termine, come la pulizia dei dati, l'addestramento dei modelli e l'ingegneria delle *feature* (*Feature Engineering*, ovvero l'estrazione e la selezione di variabili predittive a partire dai dati grezzi). In questi scenari, i requisiti e i dati stessi si evolvono continuamente durante l'esecuzione. I *framework* preesistenti generano tipicamente un piano d'azione sequenziale e statico, il quale si rivela inadeguato quando i risultati intermedi impongono un cambio di strategia. Per superare l'incapacità di adattamento dei piani sequenziali statici, è stato introdotto il paradigma del *Hierarchical Graph Modeling*, utilizzato da agenti avanzati come *Data Interpreter*.Questo approccio abbandona la rigidità delle liste lineari in favore di una scomposizione gerarchica basata su grafi diretti aciclici (DAG, strutture topologiche in cui i nodi sono connessi da archi direzionali senza formare cicli chiusi), il cui workflow operativo

è riassunto nella Figura 3.2 [3]. Analizzando lo schema nel dettaglio, è possibile notare come il processo si articoli su due livelli principali. La sezione superiore della figura illustra la modellazione gerarchica: i requisiti complessi del progetto vengono tradotti in un Grafo dei Task (Task Graph), e ciascun nodo concettuale (come l'esplorazione dei dati o l'ingegneria delle feature) viene a sua volta frammentato in un Grafo delle Azioni (Action Graph) composto da step direttamente eseguibili. La sezione inferiore evidenzia invece i tre moduli fondamentali del framework che orchestrano questo processo: il Generatore del Task Graph, il Generatore dell'Action Graph e l'Esecutore del Grafo (Graph Executor). Quest'ultimo, in particolare, gestisce l'esecuzione pratica delle singole azioni interagendo con gli strumenti esterni (Tools) e fornendo un log di feedback in tempo reale al modello linguistico, consentendo così la correzione dinamica degli errori in corso d'opera. [3].

In questa architettura, secondo la metodologia introdotta dagli autori di Data Interpreter [3], il problema principale viene scomposto dal modello linguistico in un *Task Graph* di alto livello, dove ogni singolo nodo rappresenta un macro-obiettivo (ad esempio, l'esplorazione dei dati) e gli archi definiscono le dipendenze logiche tra di essi. A sua volta, ogni macro-task viene raffinata in un *Action Graph* microscopico, in cui i nodi corrispondono a frammenti di codice eseguibile. L'innovazione metodologica di questo specifico framework risiede nella *Programmable Node Generation*: invece di invocare strumenti esterni tramite semplici chiamate API isolate, l'agente inietta dinamicamente la logica degli strumenti all'interno del codice generato. Tale processo fonde librerie standard (come `pandas` o `numpy`) con *tool* specifici recuperati in tempo reale in base alle necessità del contesto [3].

La caratteristica più dirompente dell'architettura a grafo è la sua capacità di ottimizzazione e raffinamento iterativo (*Iterative Graph Refinement*). Grazie al modulo di esecuzione integrato, l'agente monitora il flusso dei dati e l'output di ogni

singolo nodo. Qualora l'esecuzione di un blocco fallisca o la validazione dei dati non produca i risultati attesi, l'agente evita l'interruzione globale dell'esecuzione; al contrario, isola il nodo problematico, analizza l'errore attraverso meccanismi di riflessione (*Reflection*, la capacità del modello di analizzare il proprio output per autodiagnosticare errori logici) e rigenera esclusivamente la porzione del grafo che ha fallito, unitamente ai suoi discendenti diretti. Questo meccanismo preserva intatta la memoria e i risultati dei compiti precedentemente completati. Tale approccio trasforma il piano di lavoro da una struttura rigida a un'entità dinamica che si adatta in tempo reale ai mutamenti del contesto operativo (quali eccezioni durante l'esecuzione, alterazioni nei dati intermedi o l'evoluzione dei requisiti del task), garantendo una robustezza senza precedenti nella risoluzione di problemi scientifici e analiticamente complessi [3].

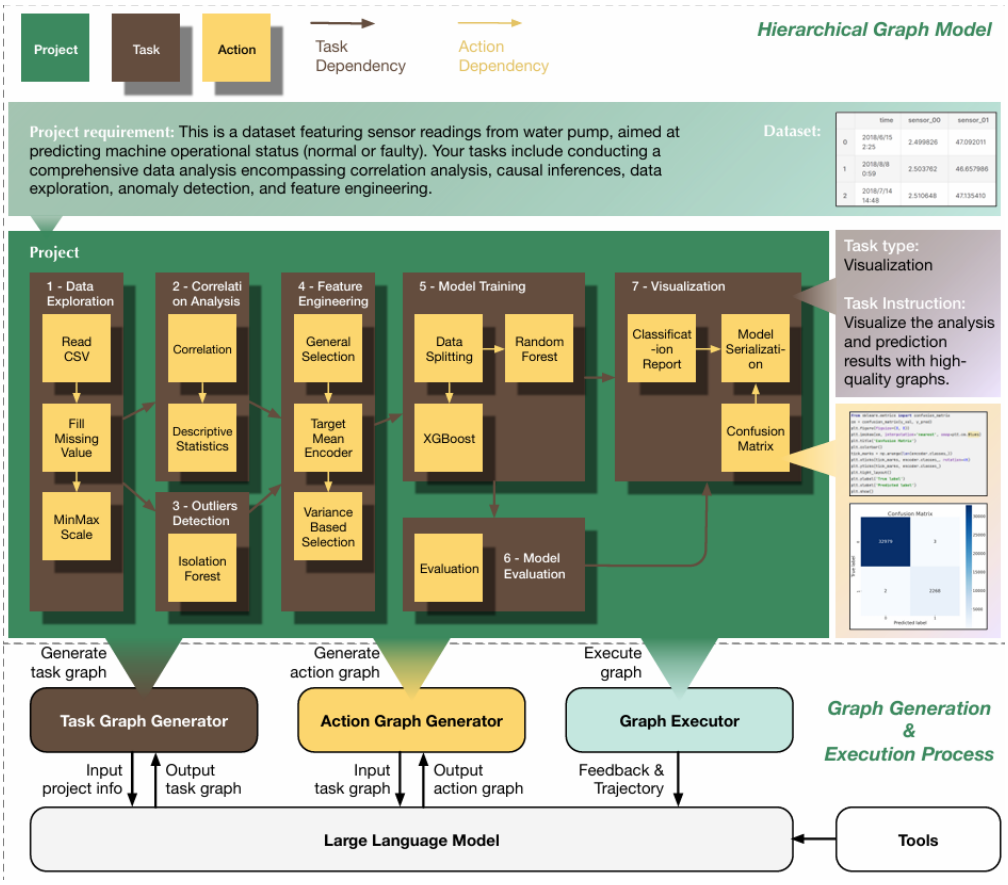


Figura 3.2: Workflow del framework Data Interpreter: modellazione gerarchica a grafo per compiti di data science (Fonte: [3])

Capitolo 4

L'Evoluzione dell'Affidabilità: Testing e Debugging Dinamico

4.1 Il Paradigma Self-Improving: L'Apprendimento senza Pesi

Nel panorama dell'automazione software, l'evoluzione dei sistemi intelligenti è paragonabile alla transizione da apparati statici a sistemi di controllo adattivi, in grado di generare autonomamente i propri strumenti di correzione. Tradizionalmente, l'addestramento dei modelli linguistici di grandi dimensioni (LLM) richiede ingenti risorse computazionali e l'aggiornamento dei pesi della rete neurale tramite meccanismi di *backpropagation* (apprendimento basato sul calcolo del gradiente), un processo matematico di ottimizzazione. Sebbene questo approccio garantisca l'acquisizione di conoscenze fondamentali, risulta impraticabile per l'adattamento in tempo reale a nuovi compiti di programmazione o per la correzione dinamica dei bug. Per superare l'inefficienza computazionale legata al *fine-tuning* (ovvero il processo di sintonizzazione fine che adatta un modello pre-addestrato a compiti

specifici modificandone i pesi interni), la ricerca recente si è orientata verso architetture di agenti in grado di attuare un apprendimento basato sull'esperienza (non basato sul gradiente), in cui l'agente modifica autonomamente il proprio codice sorgente e le proprie strategie.

Inizialmente, l'automazione avanzata si è basata su architetture duali in cui il sistema è suddiviso logicamente in due entità distinte: il *Target Agent* (l'agente esecutore, incaricato di interagire con l'ambiente, scrivere codice e utilizzare strumenti esterni come compilatori e terminali) e il *Meta Agent* (l'agente supervisore, delegato al monitoraggio delle interazioni e dei fallimenti dell'esecutore per ottimizzarne i prompt e implementare nuove routine). Tuttavia, il paradigma del *Self-Improving Coding Agent* rappresenta una successiva e cruciale innovazione architetturale che supera e annulla questa frammentazione. In questa configurazione unificata, una singola entità analizza i propri log di esecuzione e le tracce degli errori, agendo direttamente sull'intero proprio codice sorgente per estendere e perfezionare in modo olistico le proprie capacità operative [9].

L'approccio basato sull'apprendimento iterativo non supervisionato introduce, tuttavia, sfide di stabilità critiche. Nello specifico, l'aggiornamento autonomo è esposto al rischio di un degrado iterativo delle prestazioni (*performance regression loop*): una modifica del codice sub-ottimale potrebbe deteriorare le capacità preesistenti del sistema, innescando un ciclo in cui gli errori logici si accumulano e si auto-amplificano.Per mitigare questo fenomeno, l'approccio proposto nello studio sul framework SICA (Self-Improving Coding Agent) incorpora un modulo di memoria a lungo termine che funge da buffer storico. Tale soluzione permette all'agente di eseguire un *rollback*, ovvero un meccanismo sistematico di ripristino di uno stato operativo precedente e formalmente validato, qualora le modifiche apportate autonomamente non superino le rigide soglie di affidabilità imposte dai benchmark di validazione [9].

4.2 Separazione dei Ruoli e Isolamento dei Task

Nel dominio dell'ingegneria del software, un principio fondamentale impone la *Separation of Concerns* (separazione delle responsabilità) applicata alla fase di collaudo: l'entità che implementa il codice non dovrebbe coincidere con quella che ne verifica la correttezza, al fine di evitare di incorrere in fisiologici punti ciechi valutativi. Questo principio si estende in modo speculare ai sistemi basati su intelligenza artificiale. Uno dei limiti intrinseci degli agenti di generazione del codice di prima generazione (ovvero i sistemi tradizionali basati su un singolo agente, in cui un unico modello linguistico è incaricato simultaneamente sia della stesura che del testing del software) risiede proprio nella sovrapposizione dei ruoli. Quando un singolo modello linguistico è responsabile sia della sintesi del codice sorgente sia della generazione delle casistiche di test, emerge un forte *self-evaluation bias* (bias di autovalutazione). Questo fenomeno spinge l'algoritmo a produrre test tautologici e banali, progettati per accomodare e assecondare la logica, potenzialmente errata, del codice appena scritto, anziché metterne alla prova i casi limite. Il risultato è un'alta percentuale di test formalmente superati che, tuttavia, mascherano vulnerabilità critiche [5].

Per superare questa criticità, lo specifico approccio proposto dal framework cooperativo AgentCoder (la cui architettura è schematizzata nella Figura 4.1) scompone il ciclo di sviluppo introducendo un isolamento rigoroso dei task attraverso tre agenti specializzati e del tutto indipendenti [5]:

- **Programmer Agent:** Un modulo focalizzato esclusivamente sull'analisi dei requisiti e sulla sintesi del codice sorgente, privo di qualsiasi visibilità sulla successiva progettazione dei test.
- **Test Designer Agent:** Un'entità parallela che riceve in input unicamente i requisiti iniziali, senza avere alcun accesso al codice generato dal Programmer

Agent. Il suo compito è sviluppare una suite di test rigorosa, oggettiva e ad alta copertura, eliminando in radice il bias di validazione.

- **Test Executor Agent:** Questo componente agisce da modulo di orchestrazione operativo. Si occupa di compilare il codice del Programmer e di sottoporlo ai test del Designer all'interno di una *sandbox*, ovvero un ambiente di esecuzione sicuro e isolato dal sistema ospite, atto a prevenire esecuzioni di codice instabile o malevolo. Raccoglie quindi i risultati di esecuzione per restituirli sotto forma di feedback testuale e strutturato.

Il vero potenziale di questa separazione architetturale si manifesta nel ciclo di feedback iterativo. Qualora il codice fallisca, il Programmer Agent non riceve un generico avviso di errore, bensì un'analisi diagnostica dettagliata dal Test Executor Agent, comprensiva dell'input fornito, dell'output atteso e dell'output effettivamente prodotto. L'isolamento procedurale e il ciclo di feedback continuo emulano fedelmente le pratiche di *Test-Driven Development* (TDD) – una metodologia di sviluppo in cui la stesura e il fallimento dei test guidano il successivo raffinamento del codice – incrementando drasticamente la robustezza del sistema e il *Pass@1*, ovvero la metrica standard che quantifica la percentuale di problemi risolti al primo tentativo di generazione senza successivi interventi umani, superando nettamente le prestazioni dei modelli *single-agent* [5].

4.3 Debugging a Runtime: Dall'Analisi Statica all'Esecuzione Attiva

In diagnostica dei sistemi complessi, l'ispezione dello stato interno al sistema in funzione offre vantaggi inestimabili rispetto all'analisi a "scatola chiusa". In ambito software, la mera analisi statica – limitata al *parsing* del codice sorgente

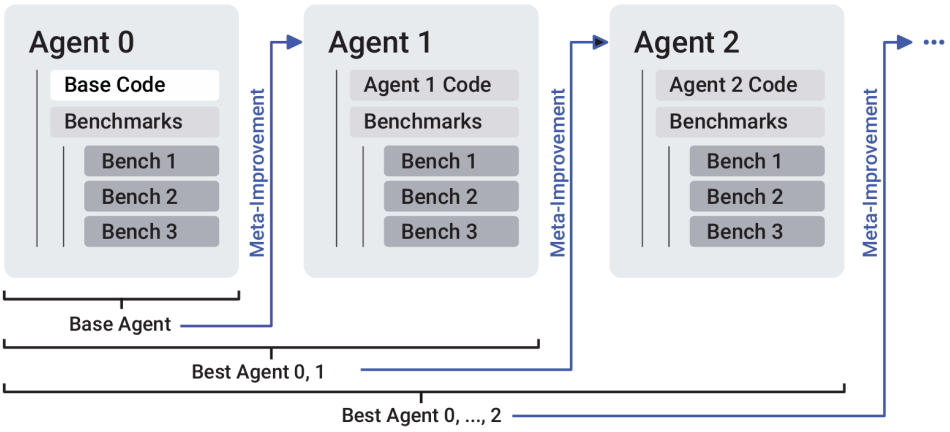


Figura 4.1: Architettura multi-agente di AgentCoder con cicli di generazione, test e ottimizzazione del codice (Fonte: [5])

inattivo – rende la correzione degli errori (*debugging*) un'operazione estremamente insidiosa per l'intelligenza artificiale. I framework standard basati su Large Language Models (LLM) tendono a trattare il programma fallito come un'entità monolitica: l'intero codice e il messaggio di errore vengono reinseriti nel modello in un singolo blocco logico. Tuttavia, per script caratterizzati da flussi di dati intricati e strutture di controllo complesse, questa tecnica si rivela profondamente inefficace. Costretto a inferire il problema senza alcuna visibilità sull'esecuzione, l'LLM è spesso portato a generare correzioni viziate da allucinazioni, alterando inavvertitamente sezioni di codice originariamente corrette [16].

Per superare questa limitazione, la ricerca ha mutuato le pratiche umane di ispezione dinamica. Il framework *Large Language Model Debugger* (LDB) [16], il cui funzionamento è illustrato nella Figura 4.2, un sistema di validazione iterativa basato sul tracciamento del flusso di esecuzione, introduce un fondamentale cambio di paradigma. Si transita così dall'analisi statica alla verifica dinamica a *runtime*, ovvero l'ispezione dello stato della memoria e dell'allocazione delle variabili durante l'effettiva esecuzione del programma. Invece di costringere il modello a processare

l'intero file simultaneamente, LDB segmenta l'algoritmo in *basic blocks* (blocchi di base). Mutuati dalla teoria dei grafi di flusso di controllo, i blocchi di base sono definiti come sequenze lineari di istruzioni dotate di un unico punto di ingresso e un unico punto di uscita [16].

All'interno di un ambiente di *testing* isolato, il sistema inserisce dei *breakpoint* (punti di interruzione) automatici al termine di ciascun blocco logico. In corrispondenza di tali istanti, il framework sospende l'esecuzione e cattura lo stato dell'ambiente operativo, registrando i valori assunti da tutte le variabili intermedie. L'agente intelligente viene quindi interrogato in modo sequenziale, blocco per blocco. Il sistema fornisce all'LLM lo scopo semantico del segmento, il codice corrispondente e lo stato delle variabili a *runtime*, richiedendo una validazione esplicita del comportamento per quella specifica porzione logica.

Questo approccio granulare abbatte drasticamente il carico cognitivo sul modello. Limitando la finestra di attenzione dell'LLM a una singola operazione, ancorata a una precisa "linea temporale" di esecuzione, l'agente acquisisce la capacità di identificare con precisione chirurgica il blocco esatto in cui i dati deviano dal comportamento atteso. Tale scomposizione consente la formulazione di *patch* di correzione localizzate, altamente deterministiche e prive di regressioni indotte [16].

4.4 Automazione della Code Review

La revisione del codice (*code review*) costituisce il processo sistematico di ispezione (*auditing*) del codice sorgente da parte di pari, finalizzato all'identificazione di bug, all'applicazione degli standard di formattazione e alla verifica della coerenza documentale. Pur essendo un presidio fondamentale per garantire l'affidabilità del software, richiede un notevole sforzo manuale. I metodi di automazione tra-

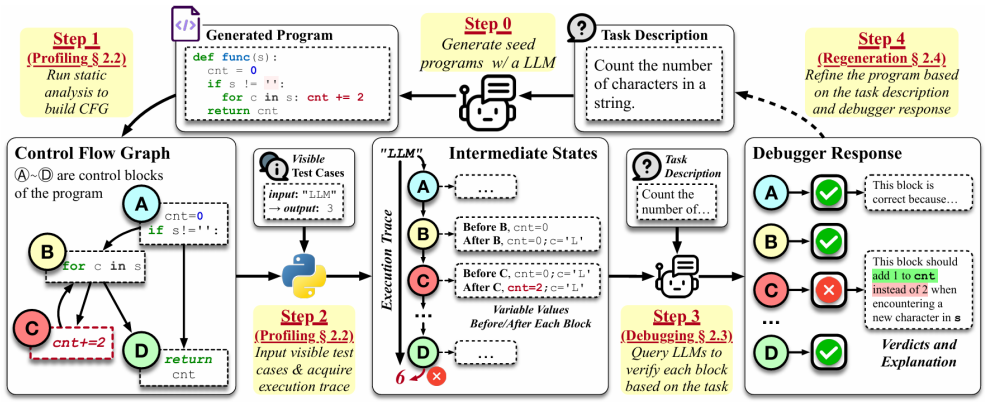


Figura 4.2: Framework LDB: segmentazione del codice in blocchi di base e verifica iterativa dell'esecuzione runtime (Fonte: [16])

dizionali, basati su modelli generativi a singolo passaggio (*input-output*), faticano intrinsecamente a emulare la natura altamente collaborativa e dialettica di tale processo. Per colmare questa lacuna, la ricerca ha introdotto framework pionieristici come *CodeAgent*, un sistema multi-agente basato su LLM progettato per simulare le dinamiche di un vero team di sviluppo, orchestrando l'interazione tra ruoli specifici quali autori delle modifiche (*code change authors*), revisori (*reviewers*) e decisori (*decision makers*) [11].

Tuttavia, l'implementazione di sistemi multi-agente basati su ragionamenti a catena – o *Chain-of-Thought* (CoT), una tecnica che scompone l'inferenza di un problema complesso in una sequenza di passaggi logici intermedi – espone a una sfida architetturale critica nota come *prompt drifting*. Questo fenomeno di deriva semantica si manifesta quando, a causa della natura stocastica dei modelli linguistici, l'accumulo di risposte e divagazioni porta il sistema a deviare progressivamente dal *task* originario. Per arginare tale vulnerabilità, CodeAgent incorpora il *QA-Checker* (*Question-Answer Checker*), un modulo di supervisione delegato esclusivamente al monitoraggio semantico della conversazione. Qualora rilevi un disallineamento tra le risposte prodotte e l'obiettivo del dialogo, il QA-Checker

interviene attivamente generando istruzioni correttive atte a riallineare il processo inferenziale verso gli scopi originari [11].

L'efficacia di questa orchestrazione è stata empiricamente dimostrata nello studio condotto da Tang et al. [11] in scenari di sviluppo reali. Il framework CodeAgent rileva accuratamente le incoerenze tra codice e messaggi di *commit*, valida la coerenza stilistica e, in particolare, identifica l'introduzione di vulnerabilità con tassi di successo nettamente superiori rispetto a modelli consolidati come GPT-4 e CodeBERT, eccellendo altresì nella generazione di suggerimenti per la risoluzione dei difetti (*bug fixing*). L'indispensabilità del supervisore è confermata dagli studi di ablazione: la rimozione del modulo QA-Checker provoca un crollo drastico nelle capacità diagnostiche della rete, dimostrando inequivocabilmente la centralità del controllo direzionale nelle dinamiche generative multi-agente [11].

4.5 Sintesi Architetturale: Un Ecosistema di Apprendimento Integrato

La sintesi delle innovazioni architetturali analizzate delinea un nuovo paradigma per l'ingegneria del software, orientato verso un ecosistema integrato e autosufficiente. L'intelligenza artificiale cessa di essere un mero generatore statistico di testo per evolversi in un'entità complessa, in grado di sintetizzare codice, collaudarlo in rigorosa assenza di *bias* confermativi, ispezionarne dinamicamente gli stati intermedi e supervisionare i propri processi inferenziali per prevenire la deriva semantica. In tale scenario, l'efficacia di un agente moderno non risiede esclusivamente nella scala parametrica del modello linguistico sottostante, bensì nella sinergia e nell'orchestrazione dei suoi moduli operativi e di verifica.

Un'architettura allo stato dell'arte prevede, in prima istanza, l'impiego di framework cooperativi quali *AgentCoder* [5], essenziali per garantire una generazione

algoritmica validata da test rigorosi e isolati. Qualora la fase di collaudo evidenziasse delle anomalie, intervengono meccanismi di ispezione a *runtime* come LDB [16]. Attraverso la segmentazione del flusso logico, tali strumenti consentono all'agente di eseguire un'ispezione blocco per blocco e di isolare con precisione chirurgica la *root cause*, ovvero il punto logico esatto in cui ha origine il difetto primigenio prima della sua propagazione a cascata.

Infine, le soluzioni diagnosticate attraverso tale processo iterativo vengono assimilate in tempo reale dall'architettura unificata del paradigma *Self-Improving* [9]. Anziché delegare la sintesi a un'entità gerarchica superiore, il sistema analizza i propri successi e fallimenti estrapolando la correzione del singolo *bug* per astrarla in una direttiva generale, aggiornando olisticamente il proprio codice base. La convergenza tra orchestrazione multi-agente, verifica dinamica e apprendimento iterativo non supervisionato trasforma di fatto l'agente in un ingegnere del software artificiale, dotato di un'evoluzione continua, autonoma e formalmente misurabile.

Capitolo 5

Il Paradosso della Produttività: Limiti e Sicurezza

5.1 Il Paradosso dei File di Contesto

Se si fornisse a un turista una mappa urbana iperdettagliata, comprensiva di ogni singolo tombino e cartello stradale, non lo si aiuterebbe a raggiungere il proprio albergo. Al contrario, si genererebbe solo un profondo disorientamento, spingendolo a vagare a vuoto tra dettagli ininfluenti e dilatando i tempi di arrivo. Una dinamica analoga si verifica nel dominio degli agenti di Intelligenza Artificiale: fornire a un modello linguistico un riassunto testuale globale e indiscriminato dell'intero progetto, con l'intento di facilitarne l'orientamento, porta spesso l'agente a esplorare percorsi logici inutili, aumentando i costi computazionali e riducendo la precisione dell'operazione.

Una pratica particolarmente diffusa nello sviluppo software assistito da IA è l'utilizzo di file di contesto a livello di repository (come il noto standard `AGENTS.md`). Questi file, fortemente raccomandati dagli sviluppatori dei principali modelli linguistici, fungono da "libretto di istruzioni" dedicato esclusivamente all'agente au-

tonomo (un esempio di architettura che sfrutta tali informazioni per la navigazione a livello di repository è visibile in Figura 5.1). Al loro interno contengono panoramiche dell'architettura di sistema, linee guida di stile, pattern di progettazione e comandi specifici per l'esecuzione dei test. L'assunto di base di questa metodologia è che, fornendo all'agente una mappa chiara della struttura del repository, l'agente stesso possa navigare il codice in modo più efficiente, commettere un numero inferiore di errori sintattici e risolvere i bug con una maggiore percentuale di successo.

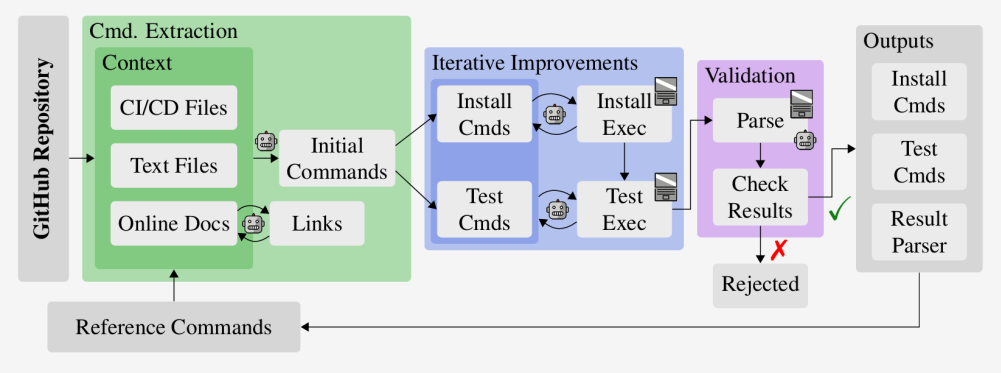


Figura 5.1: Architettura di RepoAgent: pianificazione e navigazione a livello di repository (Fonte: [12])

Tuttavia, una rigorosa analisi empirica condotta sui task di ingegneria del software reali (come i task valutati tramite il benchmark SWE-bench) dallo studio di Gloaguen et al. sull'impatto dei file di contesto [2] ha categoricamente smentito tale convinzione. I risultati dimostrano che l'inclusione di file di contesto non migliora in modo statisticamente significativo il tasso di successo degli agenti. Al contrario, la presenza di questi documenti contestuali, in particolare di quelli generati in modo automatico dagli stessi LLM, tende a ridurre le prestazioni generali. La causa di questo calo prestazionale risiede in un cambiamento comportamentale indotto proprio dall'eccesso di istruzioni fornite: leggendo il file di contesto, l'agente diventa eccessivamente scrupoloso e viene spinto a esplorare un

numero maggiore di file, eseguire test in modo inutilmente estensivo e formulare ragionamenti su percorsi del codice non strettamente necessari per la risoluzione del singolo ticket in esame.

Questo comportamento eccessivamente esplorativo ha una conseguenza diretta e misurabile sui costi operativi. L'utilizzo dei file di contesto aumenta il consumo di token di ragionamento (i frammenti di testo elaborati dal modello, che rappresentano di fatto la "moneta di scambio computazionale" spesa per generare la risposta), portando a un incremento dei costi di inferenza superiore al 20% in media per ogni singolo task. L'unica eccezione rilevata in letteratura è rappresentata dai file di contesto redatti manualmente da sviluppatori umani; questi ultimi, limitandosi a definire i requisiti minimi del progetto e le convenzioni non standardizzate (evitando quindi le ridondanti panoramiche architetturali), superano le prestazioni dei file generati automaticamente. In sintesi, l'uso indiscriminato di file di contesto rende i compiti artificialmente più complessi, dimostrando che, per guidare in modo efficiente un agente IA, l'eccesso di informazioni descrittive è spesso più dannoso della loro totale assenza [2].

5.2 Agentic Refactoring: Pulizie di Superficie o Ristrutturazione?

Ristrutturare una casa può significare due cose: dare una passata di vernice fresca e cambiare le maniglie delle porte, oppure abbattere un muro per unire due stanze. Nel primo caso il rischio è nullo, nel secondo è indispensabile l'intervento di un ingegnere. L'Intelligenza Artificiale, quando viene lasciata libera di "sistemare" il codice, si comporta oggi come un imbianchino eccezionalmente rapido nel lucidare la superficie e riordinare la nomenclatura, ma che fatica enormemente a toccare i muri portanti del software.

Il refactoring è un'attività fondamentale per la sostenibilità del software, finalizzata a migliorare la qualità interna del codice senza alterarne il comportamento osservabile. Con l'avvento dei *coding agent* autonomi, questi ultimi sono diventati partecipanti attivi in tale processo di manutenzione. Un'analisi su larga scala di migliaia di pull request (le richieste formali di integrazione delle modifiche nel codice sorgente) condotta nello studio sull'Agentic Refactoring di Horikawa et al. [4] rivela che il refactoring è un'attività comune e intenzionale in questo nuovo paradigma di sviluppo. Gli agenti effettuano operazioni di refactoring in circa il 26% dei *commit* generati (i pacchetti di salvataggio dei singoli interventi). Le motivazioni principali che guidano l'intervento dell'agente riguardano in modo schiacciante attributi di qualità interna (ovvero quelle caratteristiche strutturali che migliorano il software senza alterarne il comportamento osservabile dall'esterno), concentrandosi per oltre l'80% sul miglioramento della manutenibilità e della leggibilità del codice.

Analizzando la tipologia degli interventi, emerge una netta discrepanza tra il refactoring eseguito dall'IA e quello tipico degli sviluppatori umani. Tale divergenza fa sì che l'azione degli agenti sia dominata da modifiche di basso livello, orientate alla coerenza sintattica e stilistica. Le operazioni più frequenti includono il cambio di tipo delle variabili, la rinomina dei parametri e la rinomina delle variabili locali. Queste operazioni dimostrano una chiara preferenza dell'Intelligenza Artificiale per miglioramenti localizzati, limitati ai blocchi interni di codice, i quali non richiedono una comprensione olistica del sistema. Al contrario, le ristrutturazioni di alto livello che alterano le firme dei metodi, l'architettura delle classi o mirano all'eliminazione delle duplicazioni (attività comuni nel refactoring umano) risultano significativamente sottorappresentate.

La preferenza dell'Intelligenza Artificiale per le modifiche superficiali ha un impatto diretto sulla reale evoluzione qualitativa del software. Se valutato attraverso

metriche strutturali, l'*agentic refactoring* produce miglioramenti statisticamente significativi ma di entità marginale, riuscendo a ridurre leggermente la dimensione e la complessità delle classi solo per interventi di medio livello. Quando si tratta di ridurre i *code smell* (sintomi o indicatori strutturali di inefficienze profonde nel design del software, come difetti di progettazione o modularizzazione insufficiente), l'impatto degli agenti è pressoché nullo. Questa inefficacia sui difetti complessi indica oggettivamente che, allo stato attuale, gli agenti operano come eccellenti revisori sintattici, ma non sono ancora in grado di agire come architetti del software capaci di pianificare trasformazioni strutturali profonde [4].

5.3 L'Illusione del Prompting per la Sicurezza

Se la porta del caveau di una banca è guasta, affiggere cartelli con la scritta "Vietato Entrare" o intimare ai passanti di prestare attenzione non renderà l'edificio più sicuro; la sicurezza si otterrà esclusivamente riparando la serratura. Nello sviluppo software assistito dall'Intelligenza Artificiale, manipolare la richiesta (il *prompt*) istruendo il modello a "scrivere codice sicuro e prestare attenzione" equivale ad apporre tali cartelli: questa pratica modificherà l'approccio generativo dell'agente, ma non ridurrà in alcun modo la quantità di vulnerabilità che il modello linguistico finirà per iniettare nel programma.

Uno dei problemi più critici nell'adozione dei *Large Language Models* (LLM) per la generazione automatica di codice è la frequente introduzione di vulnerabilità di sicurezza (quali crittografia debole, convalida impropria degli input e gestione insicura della memoria). Nel settore esiste la convinzione diffusa che l'ottimizzazione del *prompt*, ovvero l'impiego di strategie di istruzione più sofisticate, possa risolvere questa problematica guidando il modello verso risultati più robusti. Per testare la reale efficacia di tale ipotesi, recenti studi in letteratura hanno

introdotto una strategia avanzata chiamata *Weaknesses-Aware Chain-of-Thought* (WA-0CoT). Questa architettura è progettata per arricchire il *prompt* iniziale, integrando un contesto di sicurezza basato sulle mappature del framework CWE (*Common Weakness Enumeration*, un sistema di classificazione e mappatura standardizzato a livello internazionale delle vulnerabilità del software). L'obiettivo è costringere l'agente autonomo a ragionare in modo proattivo sulle potenziali falle di sicurezza prima di procedere alla generazione dell'output.

Tuttavia, un'estesa valutazione empirica condotta su molteplici linguaggi di programmazione (Java, Python, C, C++) e su diversi modelli allo stato dell'arte nello studio sull'efficacia del prompting di sicurezza di Kharma et al. [6] ha rivelato una realtà profondamente controintuitiva. I test statistici hanno dimostrato che le tecniche di *prompt engineering*, inclusa la strategia WA-0CoT, non producono alcuna riduzione statisticamente significativa nella frequenza complessiva delle vulnerabilità, né tantomeno nella loro densità (ossia il rapporto tra il numero di falle e le linee di codice generate). In altre parole, la sola ingegneria del *prompt* risulta strutturalmente insufficiente per abbassare il livello di rischio assoluto del codice generato.

Ciò che l'ottimizzazione del *prompt* altera in modo sistematico non è la sicurezza complessiva del codice, bensì la mera distribuzione compositiva delle vulnerabilità. Fornire all'agente IA un contesto di sicurezza modifica la topologia degli errori: riduce l'incidenza di alcune specifiche categorie di CWE, ma ne incrementa inevitabilmente altre, innescando effetti collaterali che variano profondamente in base al linguaggio di programmazione impiegato. Tali risultati dimostrano oggettivamente che la sicurezza del codice non può essere garantita semplicemente formulando richieste migliori o più dettagliate al modello linguistico. Al contrario, è richiesta un'attenta progettazione di *prompt* che siano consapevoli delle specificità del singolo linguaggio, i quali devono essere imperativamente affiancati da

strumenti operativi di analisi statica e validazione automatica a posteriori [6].

5.4 L'Eredità dell'IA: Debito Tecnico e Manutenzione

Immaginiamo un muratore instancabile che costruisce case a velocità record. Sul momento tutto sembra perfetto e il proprietario è soddisfatto; tuttavia, anni dopo i tubi iniziano a perdere, l'impianto elettrico va in cortocircuito e i muri mostrano crepe, poiché il muratore, per fare in fretta, ha utilizzato materiali scadenti e scorciatoie non documentate. Sorge spontanea una domanda: chi sosterrà il costo delle continue riparazioni? Allo stesso modo, l'uso massiccio dell'Intelligenza Artificiale per scrivere codice rischia oggi di lasciarci in eredità esattamente questo scenario: enormi "città" di codice che funzionano all'apparenza, ma che in futuro richiederanno costi di manutenzione spropositati.

Mentre l'adozione degli assistenti di codifica IA aumenta vertiginosamente la produttività a breve termine, si sa ancora molto poco sull'impatto reale che il codice sintetico ha una volta introdotto nei repository di produzione (gli archivi definitivi in cui risiede il software distribuito agli utenti). L'integrazione di codice generato artificialmente all'interno di progetti *open-source* su larga scala ha iniziato a mostrare le prime crepe sotto forma di debito tecnico, definibile come il costo implicito delle rielaborazioni future causato dalla scelta di implementare soluzioni rapide ma non ottimali. Un'analisi empirica condotta su centinaia di migliaia di commit creati da intelligenze artificiali nello studio di Liu et al. sul debito tecnico introdotto dall'IA [7] rivela che oltre il 15% dei commit autogenerati introduce difetti immediati. La stragrande maggioranza di queste anomalie (quasi il 90%) è costituita da *code smell*, ovvero sintomi strutturali di una progettazione subottimale che, pur non bloccando l'esecuzione immediata del software, ne degradano

gravemente la manutenibilità. Uno dei dati più allarmanti è la persistenza di questi errori: quasi un quarto (22,7%) dei difetti introdotti dall'IA sopravvive inosservato fino alle versioni più recenti dei repository, accumulandosi così silenziosamente nel tempo.

La gestione a lungo termine di questo debito tecnico evidenzia un netto squilibrio tra la velocità di generazione dell'IA e la capacità di mantenimento umano. I file generati dagli agenti autonomi, pur essendo creati rapidamente, ricevono una manutenzione molto meno frequente rispetto al codice scritto da umani e le modifiche successive tendono a interessare solo frazioni minime del documento. Inoltre, emerge una marcata asimmetria nelle tipologie di intervento. Mentre gli aggiornamenti apportati dall'Intelligenza Artificiale si concentrano quasi esclusivamente sull'estensione delle funzionalità, le operazioni critiche di correzione degli errori (*bug fixing*) ricadono, in stragrande maggioranza, sulle spalle degli sviluppatori umani.

I dati empirici finora raccolti delineano uno scenario in cui l'agente autonomo eccelle nella fase generativa iniziale, ma fallisce nel garantire la sostenibilità del ciclo di vita del software. Il codice generato dall'IA rischia quindi di trasformarsi in un onere a lungo termine per i team di sviluppo, evidenziando l'urgenza di integrare controlli di qualità (*Quality Assurance*) molto più rigorosi per prevenire l'accumulo di difetti silenti, prima che il debito di manutenzione diventi insostenibile [7, 10].

Capitolo 6

Il Problema della Valutazione (Benchmarking)

6.1 I Limiti dei Test Attuali: Contaminazione e Irrealtà

Valutare un sistema complesso esclusivamente in un ambiente di test iper-strutturato ne certifica le prestazioni teoriche, ma non offre alcuna garanzia sulla sua affidabilità in scenari operativi non strutturati. Allo stesso modo, testare gli agenti basati su Intelligenza Artificiale per la generazione di codice esclusivamente su problemi storici e predefiniti genera un'illusione di competenza; tale illusione subisce un drastico calo prestazionale non appena questi strumenti vengono messi alla prova in scenari di sviluppo del mondo reale.

Lo sviluppo di agenti autonomi richiede metriche di valutazione affidabili per misurare i reali progressi tecnologici. Recentemente, framework di valutazione come SWE-bench (le cui dinamiche operative sono ampiamente descritte nello studio di Vergopoulos et al. [12]) sono diventati lo standard di riferimento. SWE-bench

è un ambiente di test progettato per misurare la capacità degli agenti di risolvere *issue* (segnalazioni di bug o richieste di nuove funzionalità) reali, estratte da repository ospitati su GitHub, fornendo all'agente l'intero codice sorgente del progetto come contesto operativo. Tuttavia, la costruzione di benchmark a livello di repository richiede un notevole sforzo manuale per configurare ambienti di esecuzione storicamente accurati, necessari per avviare le suite di test originali e validare le patch generate.

A causa di questo severo limite operativo legato all'intervento umano, i framework attuali tendono a concentrarsi su un numero estremamente ristretto di repository (ad esempio, solo 12 progetti per la versione standard di SWE-bench). Valutare l'intelligenza artificiale su un campione così limitato crea una grave *discrepanza distributiva* (o *distributional mismatch*): le caratteristiche dei dati utilizzati per la valutazione non coincidono con la complessità e la variabilità del software quotidiano, rendendo le prestazioni misurate scarsamente rappresentative.

Inoltre, trattandosi di progetti open-source statici e ampiamente diffusi, è altamente probabile che le soluzioni a tali bug siano già state assimilate dai modelli linguistici di grandi dimensioni durante la loro fase di addestramento. Questa esposizione pregressa causa una grave contaminazione dei dati, portando a un fenomeno di overfitting (ovvero un adattamento eccessivo del modello ai dati di addestramento, che ne compromette la capacità di generalizzare e risolvere problemi nuovi): il modello restituisce la corretta risoluzione del bug non per via di reali capacità di ragionamento logico-deduttivo autonomo, ma per aver "memorizzato" i dati originali, fornendo così una falsa metrica di intelligenza [12].

6.2 Generazione Automatica di Scenari di Valutazione

Se la progettazione manuale di un singolo benchmark richiede uno sforzo operativo considerevole, la soluzione per garantire la continuità nella valutazione delle capacità dei modelli risiede nella creazione di sistemi intelligenti capaci di progettare, assemblare e validare automaticamente nuovi ambienti di test. Per superare le limitazioni intrinseche dei benchmark statici, la ricerca si è orientata verso la generazione automatizzata di scenari di valutazione. Un'architettura innovativa a tal fine è rappresentata da SETUPAGENT, un sistema completamente autonomo basato su modelli linguistici (LLM), progettato per estrarre e sintetizzare informazioni da repository software reali. Invece di richiedere la configurazione manuale da parte di un operatore umano, SETUPAGENT naviga autonomamente nel progetto, identificando e deducendo i comandi di installazione necessari, configurando le dipendenze per garantire una riproducibilità storica accurata e predisponendo l'esecuzione delle suite di test. Nello specifico, il suo ciclo operativo si articola in tre fasi: l'estrazione iniziale dei comandi (analizzando file come README o configurazioni CI/CD), un ciclo di miglioramento iterativo (in cui l'agente esegue i comandi in un ambiente isolato e si autocorregge analizzando i log di errore) e la validazione finale dei risultati. Tale approccio trasforma istantaneamente ogni *pull request* o *issue* di GitHub in un compito di valutazione eseguibile e verificabile, permettendo la costruzione di benchmark a larga scala, privi di errori umani e pienamente scalabili. Tuttavia, l'efficacia di questo strumento presenta limiti strutturali. L'analisi dei fallimenti del framework rivela che SETUPAGENT incontra gravi criticità nei progetti in cui il collaudo richiede l'esecuzione di container Docker annidati (attualmente non supportati) o l'utilizzo di complessi *Makefile*. Inoltre, l'agente fatica a dedurre le procedure corrette quando le istruzioni di instal-

lazione richiedono una navigazione web profonda attraverso molteplici link esterni [12].

6.3 Verso Benchmark Dinamici: La Realtà delle Prestazioni

L'integrazione di dataset generati in modo completamente automatizzato (ovvero assemblando continuamente nuovi task dalle piattaforme di sviluppo reali, al fine di prevenire la contaminazione dei dati di test), basati su progetti software eterogenei e complessi, ha permesso di sottoporre i modelli di linguaggio ad analisi rigorose in condizioni operative reali, evidenziando il divario esistente tra le prestazioni teoriche degli agenti e la loro effettiva maturità per la produzione su larga scala. Attraverso l'automazione garantita da framework come SETUPAGENT, è stata resa possibile la creazione di benchmark di nuova generazione: SWEE-Bench, che estende la copertura a centinaia di repository Python diversificati per incrementare la variabilità statistica, e SWA-Bench, che sposta il focus valutativo dalle singole librerie alle applicazioni software complete.

Il confronto delle prestazioni su tali dataset, condotto dagli stessi autori del framework [12], ha rivelato differenze distribuzionali critiche rispetto allo standard SWE-bench originale. All'interno di questi nuovi benchmark generati automaticamente (come SWA-Bench e SWEE-Bench), gli agenti si confrontano con descrizioni delle *issue* meno dettagliate e architetture applicative intrinsecamente più complesse, portando a una contrazione significativa del tasso di successo, che risulta inferiore fino al 40% rispetto ai test tradizionali. Tali evidenze dimostrano inequivocabilmente che i benchmark statici e manuali tendono a sovrastimare le capacità degli agenti; pertanto, la generazione automatica e continua di test

diversificati costituisce l'unico paradigma affidabile per misurare e indirizzare lo sviluppo dell'ingegneria del software guidata dall'Intelligenza Artificiale [12].

Capitolo 7

Conclusioni

L'evoluzione dell'ingegneria del software sta attraversando un punto di svolta critico. Siamo passati rapidamente da semplici strumenti di autocompletamento del codice a veri e propri agenti autonomi basati su Large Language Models, capaci di pianificare, navigare all'interno di repository complessi, utilizzare strumenti esterni e proporre modifiche strutturate. Questa tesi ha esplorato a fondo questa transizione, analizzando non solo l'architettura e le promesse di queste nuove tecnologie, ma soprattutto il loro impatto reale in ambienti di produzione.

L'analisi dei framework moderni, come SWE-agent e CodeAgent, ha dimostrato che dotare i modelli linguistici di interfacce dedicate (Agent-Computer Interfaces) e di una pianificazione a grafo dinamica permette di superare i limiti dei vecchi approcci sequenziali. Questi agenti non si limitano a scrivere righe di codice isolate, ma simulano il processo collaborativo umano: leggono la documentazione, ispezionano i file, generano test e correggono autonomamente i propri errori attraverso cicli di feedback.

Tuttavia, l'indagine empirica condotta ha fatto emergere quello che possiamo definire il *paradosso della produttività e della sicurezza*. Se da un lato gli agenti IA accelerano drasticamente la risoluzione di bug e la scrittura di codice, dall'al-

tro operano prevalentemente a livello superficiale. Le operazioni di refactoring condotte in autonomia si concentrano quasi esclusivamente su aspetti stilistici e sintattici, ignorando i difetti di progettazione architetturale. Inoltre, l'illusione che un prompt engineering più sofisticato possa eliminare le vulnerabilità si è scontrata con l'evidenza dei dati: le falle di sicurezza non diminuiscono in modo significativo, ma cambiano semplicemente forma, accumulandosi silenziosamente come debito tecnico all'interno dei repository.

Questo ci porta al problema fondamentale della valutazione. Come abbiamo visto, i benchmark attuali, statici e costruiti manualmente su progetti eccessivamente noti, tendono a sovrastimare enormemente le reali capacità di questi agenti a causa della contaminazione dei dati di addestramento. Solo attraverso la generazione automatica e dinamica di ambienti di test su repository reali e meno conosciuti possiamo misurare le vere abilità di problem-solving dell'intelligenza artificiale, scoprendo che le loro performance nel mondo reale sono ancora limitate.

In conclusione, l'intelligenza artificiale applicata all'ingegneria del software rappresenta oggi uno strumento di potenziamento formidabile, assimilabile a uno sviluppatore *junior* instancabile e velocissimo nel gestire compiti localizzati. Tuttavia, non è ancora pronta per assumere il ruolo di architetto del software. La supervisione umana, unita a rigorosi processi di validazione, analisi statica e test dinamici, rimane un requisito imprescindibile per garantire che l'esplosione di produttività generata dall'IA non si trasformi in un insostenibile onere di manutenzione per il futuro.

Bibliografia

[1] Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. A Survey on Code Generation with LLM-based Agents, September 2025.

[2] Thibaud Gloaguen, Niels Mündler, Mark Müller, Veselin Raychev, and Martin Vechev. Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?, February 2026.

[3] Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Chenxing Wei, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, Li Zhang, Lingyao Zhang, Min Yang, Mingchen Zhuge, Taicheng Guo, Tuo Zhou, Wei Tao, Xiangru Tang, Xiangtao Lu, Xiawu Zheng, Xinbing Liang, Yaying Fei, Yuheng Cheng, Zhibin Gou, Zongze Xu, and Chenglin Wu. Data Interpreter: An LLM Agent For Data Science, October 2024.

[4] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. Agentic Refactoring: An Empirical Study of AI Coding Agents, November 2025.

[5] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation, May 2024.

[6] Mohammed Kharma, Ahmed Sabbah, Mohammad Alkhanafseh, Mohammad Hammoudeh, and David Mohaisen. An Empirical Evaluation of LLM-Generated Code Security Across Prompting Methods, May 2026.

[7] Yue Liu, Ratnadira Widyasari, Yanjie Zhao, Ivana Clairine Irsan, Junkai Chen, and David Lo. Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild, April 2026.

[8] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative Agents for Software Development, June 2024.

[9] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A Self-Improving Coding Agent, May 2025.

[10] Shota Sawada, Tatsuya Shirai, Yutaro Kashiwa, Ken'ichi Yamaguchi, Hiroshi Iwata, and Hajimu Iida. To What Extent Does Agent-generated Code Require Maintenance? An Empirical Study, May 2026.

[11] Xunzhu Tang, Kisub Kim, Yewei Song, Cedric Lothritz, Bei Li, Saad Ezzini, Haoye Tian, Jacques Klein, and Tegawende F. Bissyande. CodeAgent: Autonomous Communicative Agents for Code Review, September 2024.

[12] Konstantinos Vergopoulos, Mark Niklas Müller, and Martin Vechev. Automated Benchmark Generation for Repository-Level Coding Tasks, March 2025.

[13] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei

Wei, and Ji-Rong Wen. A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*, 18(6):186345, December 2024.

[14] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, November 2024.

[15] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges, August 2024.

[16] Li Zhong, Zilong Wang, and Jingbo Shang. Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step-by-step, June 2024.